OOP Concepts Study Guide - Java

Polymorphism

What is Polymorphism?

Polymorphism is the ability of objects of different classes to be treated as objects of a common base class, while still maintaining their own specific behaviors. The word "polymorphism" comes from Greek, meaning "many forms."

Types of Polymorphism

1. Compile-time Polymorphism (Method Overloading)

Method overloading allows multiple methods with the same name but different parameters in the same class.

```java
class Calculator {
    // Method with 2 integer parameters
    public int add(int a, int b) {
        return a + b;
    }

    // Overloaded method with 3 integer parameters
    public int add(int a, int b, int c) {
        return a + b + c;
    }

    // Overloaded method with 2 double parameters
    public double add(double a, double b) {
        return a + b;
    }
}

public class TestOverloading {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(5, 3));        // Output: 8
        System.out.println(calc.add(5, 3, 2));     // Output: 10
        System.out.println(calc.add(5.5, 3.2));    // Output: 8.7
    }
}
```

2. Runtime Polymorphism (Method Overriding)

Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its parent class.

```java
// Parent class
class Animal {
    public void makeSound() {
        System.out.println("The animal makes a sound");
    }

    public void sleep() {
        System.out.println("The animal sleeps");
    }
}

// Child class 1
class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("The dog barks: Woof! Woof!");
    }
}

// Child class 2
class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("The cat meows: Meow! Meow!");
    }
}

public class TestPolymorphism {
    public static void main(String[] args) {
        Animal myAnimal;

        myAnimal = new Dog();
        myAnimal.makeSound();  // Output: The dog barks: Woof! Woof!

        myAnimal = new Cat();
        myAnimal.makeSound();  // Output: The cat meows: Meow! Meow!

        myAnimal.sleep();      // Output: The animal sleeps
    }
}
```

Key Points about Polymorphism

- **Dynamic Method Dispatch**: The JVM determines which method to call at runtime based on the actual object type
- **IS-A Relationship**: Polymorphism works through inheritance relationships
- **Method Signature**: Overloaded methods must have different parameter lists
- **@Override Annotation**: Good practice to use when overriding methods

Benefits of Polymorphism

- **Code Reusability**: Write code once, use with multiple object types
- **Flexibility**: Easy to add new classes without modifying existing code
- **Maintainability**: Changes to specific implementations don't affect client code

Abstraction

What is Abstraction?

Abstraction is the process of hiding implementation details while showing only the essential features of an object. It focuses on what an object does rather than how it does it.

Ways to Achieve Abstraction in Java

1. Abstract Classes

An abstract class cannot be instantiated and may contain both abstract and concrete methods.

java

```java
// Abstract class
abstract class Shape {
    protected String color;

    // Constructor
    public Shape(String color) {
        this.color = color;
    }

    // Abstract method (must be implemented by subclasses)
    public abstract double calculateArea();

    // Concrete method (can be inherited as-is)
    public void displayColor() {
        System.out.println("Color: " + color);
    }
}

// Concrete class extending abstract class
class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    private double length, width;

    public Rectangle(String color, double length, double width) {
        super(color);
        this.length = length;
        this.width = width;
    }

    @Override
    public double calculateArea() {
        return length * width;
    }
```

```java
}

public class TestAbstraction {
    public static void main(String[] args) {
        Shape circle = new Circle("Red", 5.0);
        Shape rectangle = new Rectangle("Blue", 4.0, 6.0);

        circle.displayColor();
        System.out.println("Circle Area: " + circle.calculateArea());

        rectangle.displayColor();
        System.out.println("Rectangle Area: " + rectangle.calculateArea());
    }
}
```

2. Interfaces

An interface defines a contract that implementing classes must follow. All methods in an interface are implicitly public and abstract (before Java 8).

java

```java
// Interface
interface Drawable {
    void draw();  // implicitly public and abstract
    void resize(double factor);
}

interface Colorable {
    void setColor(String color);
    String getColor();
}

// Class implementing multiple interfaces
class Square implements Drawable, Colorable {
    private double side;
    private String color;

    public Square(double side) {
        this.side = side;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a square with side: " + side);
    }

    @Override
    public void resize(double factor) {
        side *= factor;
        System.out.println("Square resized. New side: " + side);
    }

    @Override
    public void setColor(String color) {
        this.color = color;
    }

    @Override
    public String getColor() {
        return color;
    }
}

public class TestInterface {
    public static void main(String[] args) {
        Square square = new Square(5.0);
        square.setColor("Green");
```

```
        square.draw();
        square.resize(1.5);
        System.out.println("Color: " + square.getColor());
    }
}
```

Abstract Class vs Interface

Abstract Class	Interface
Can have both abstract and concrete methods	All methods are abstract (before Java 8)
Can have constructors	Cannot have constructors
Can have instance variables	Only constants (public static final)
Uses `extends` keyword	Uses `implements` keyword
Single inheritance	Multiple inheritance supported
Can have access modifiers	Methods are implicitly public

Real-world Example

java

```java
// Abstract class representing a vehicle
abstract class Vehicle {
    protected String brand;
    protected String model;

    public Vehicle(String brand, String model) {
        this.brand = brand;
        this.model = model;
    }

    // Abstract methods
    public abstract void start();
    public abstract void stop();
    public abstract double getFuelEfficiency();

    // Concrete method
    public void displayInfo() {
        System.out.println("Brand: " + brand + ", Model: " + model);
    }
}

// Interface for vehicles that can be charged
interface Chargeable {
    void charge();
    int getBatteryLevel();
}

// Concrete implementation
class ElectricCar extends Vehicle implements Chargeable {
    private int batteryLevel;

    public ElectricCar(String brand, String model) {
        super(brand, model);
        this.batteryLevel = 100;
    }

    @Override
    public void start() {
        System.out.println("Electric car started silently");
    }

    @Override
    public void stop() {
        System.out.println("Electric car stopped");
    }
```

```java
    @Override
    public double getFuelEfficiency() {
        return 120.0; // miles per charge
    }

    @Override
    public void charge() {
        batteryLevel = 100;
        System.out.println("Car fully charged");
    }

    @Override
    public int getBatteryLevel() {
        return batteryLevel;
    }
}
```

Benefits of Abstraction

- **Security**: Hides internal implementation details
- **Code Organization**: Separates interface from implementation
- **Maintenance**: Changes to implementation don't affect client code
- **Standardization**: Ensures consistent method signatures across implementations

---

Quiz Questions

1. Which type of polymorphism is resolved at compile time? a) Method overriding b) Method overloading c) Dynamic binding d) Interface implementation

Correct Answer: b) Method overloading

2. What keyword is used to implement an interface in Java? a) extends b) implements c) interface d) abstract

Correct Answer: b) implements

3. Which of the following statements about abstract classes is FALSE? a) Abstract classes cannot be instantiated b) Abstract classes can have constructors c) Abstract classes must have at least one abstract method d) Abstract classes can have concrete methods

Correct Answer: c) Abstract classes must have at least one abstract method

4. In method overriding, which annotation is recommended to use? a) @Deprecated b) @Override c) @SuppressWarnings d) @Abstract

Correct Answer: b) @Override

5. How many interfaces can a class implement in Java? a) Only one b) Maximum two c) Maximum five d) Multiple (no limit)

Correct Answer: d) Multiple (no limit)

Practice Coding Questions

Question 1: Create an abstract class `Employee` with the following requirements:

- Abstract method `calculateSalary()` that returns a double
- Concrete method `displayInfo()` that prints employee name and ID
- Constructor that takes name and ID as parameters
- Create two concrete classes `FullTimeEmployee` and `PartTimeEmployee` that extend `Employee`
- `FullTimeEmployee` should calculate salary as fixed monthly amount
- `PartTimeEmployee` should calculate salary as hourly rate × hours worked

Question 2: Design an interface `MediaPlayer` with methods `play()`, `pause()`, `stop()`, and `setVolume(int volume)`. Create two classes `AudioPlayer` and `VideoPlayer` that implement this interface. Each should have different implementations showing what type of media they're playing.

Question 3: Demonstrate method overloading by creating a class `MathOperations` with multiple `multiply` methods:

- `multiply(int a, int b)` - multiplies two integers
- `multiply(double a, double b)` - multiplies two doubles
- `multiply(int a, int b, int c)` - multiplies three integers
- `multiply(int[] numbers)` - multiplies all numbers in an array

Question 4: Create an abstract class `BankAccount` with abstract methods `deposit(double amount)` and `withdraw(double amount)`. Include a concrete method `getBalance()`. Create two concrete classes `SavingsAccount` and `CheckingAccount` with different withdrawal rules (SavingsAccount has a minimum balance requirement, CheckingAccount allows overdraft up to a limit).

Question 5: Design a polymorphism example with a base class `Instrument` and derived classes `Piano`, `Guitar`, and `Drums`. Each should override a `playSound()` method. Create an array of `Instrument` objects and demonstrate polymorphic behavior by calling `playSound()` on each element.